TopoGym: Library Overview and Environment Specifications

Jason Carlson

August 2026

Scope. This document specifies the TopoGym environment library in depth: the base benchmark (**GridWorld2D**), its textured variant (**GridWorld2D-Texture**), and its topological variant (**GridWorld2D-Top**, non-trivial global topology via edge identifications). Together the three variants form one benchmark, **TopoGym-v1**: the versioned, pinned registry across all variants under the universal action and observation spaces of Section 1.11 — Plain, Texture, and Top are reporting slices of a single suite, and the version pins the environment list (future families extend to v2 without altering v1 entries). This document is the authority on the environments themselves, and Section 4 documents the library's code interfaces with usage examples.

1 GridWorld2D

A registry of named environments backed by one parametric generator. Everything is generated from a frozen configuration and a seed; nothing is hand-placed. The design goals: ground truth matches the theory's objects exactly, so mechanism predictions and certificates are checkable programmatically; and the interface follows existing gym-library conventions — stable named environments with seed variation and a few modifiable parts — so the benchmark is directly usable by others.

1.1 World model

A `size` × `size` grid of cells, partitioned into free and obstacle cells. The agent occupies one free cell; actions are {up, down, left, right}; moving into an obstacle leaves the agent in place. Standing conventions, enforced at generation time:

- the exterior free space (the component containing the start cell) is 4-connected;
- obstacles are well-composed: no 2×2 window contains exactly a diagonal pair of obstacle cells (no diagonal pinch);
- every door has width one: a free cell with obstacle cells on two opposite sides and free cells on the other two;
- a chamber is an obstacle wall enclosing free interior once its doors are sealed; a decoy is a sealed ring enclosing no reachable free cell — its interior is unvisitable by construction, so persistence against it is never rewarded.

1.2 Registry: named environments (the primary interface)

Following the convention of existing gym libraries, TopoGym's canonical interface is a registry of named, pre-defined environments: `gym.make("TopoGym/{Family}-{size}-v0", seed=`n`)`. Each registry entry is a frozen configuration of the underlying generator; the seed drives layout variation (placement, door positions, shape assignment) within that configuration; and a small set of documented parts remains modifiable as `make` kwargs (`p_slip`, `reward_mode`, and per-family knobs noted below). The benchmark is the registry crossed with the published seed lists.

Family	Description	Sizes / variants
`Dilution-{50,200}`	one chamber, no decoys	kwarg: `chamber_side`
`Chambers2-{50,100,200,400}`	two chambers, fixed geometry; the world-scaling family	—
`ChamberCount{1,2,4,8}-200`	k separated chambers	—
`Decoys{0,1,2,4,8}-50`	one chamber among k_d sealed decoys	kwarg: `decoy_side`
`Shape{Sq,Ci,Tr,St}-50`	area-matched chamber shapes, centered, bottom-left start	kwarg: `chamber_side`
`Nested{1,2,3}-50`	sequentially nested shells	kwarg: `doors_per_chamber`
`GiveUp{1,2,4}-50`	door behind a dead-end corridor of the given length, centered, bottom-left start	kwarg: `door_corridor_len`
`Bottleneck{3,6}-100`	centered tree of side-24 rooms joined by width-1 corridors of the given length	kwargs: `rooms`, `corridor_len`, `chamber_side`
`Maze-{50,100}`	seeded perfect maze (uniform spanning tree over cells)	kwarg: `braid`

Registry ids are stable across releases; new families extend the registry rather than altering existing entries.

1.3 Generator API (advanced)

Under the registry sits the full parametric generator (`topogym.generation.TopoGenConfig2D`), available to power users and used to define registry entries. Its configuration schema:

- `size` — grid side length; any integer accepted.
- `n_chambers`, `n_decoys`; `chamber_side`, `decoy_side`; `chamber_shape`, `decoy_shape` ({square, circle, triangle, star, mixed}, area-matched at equal `side` so shape is never confounded with size).
- `min_sep` — minimum pairwise Chebyshev wall separation, enforced at placement with an up-front packing feasibility check; configurations claiming the one-entry-per-rollout property must satisfy `min_sep` $> L$ (rollout horizon), recorded per configuration in the manifest.
- `doors_per_chamber` (default 1); `door_kind` (`open` — a visible walkable width-1 doorway, the registry convention — or `bump`, hidden until opened by repeated bumps); door width fixed at 1 and validated; `door_corridor_len` (default 0), a dead-end corridor outside each door controlling entry probability q.
- `style` — `open` (alias `rooms`), `nested` (`nested_depth`, `shell_spacing`), `corridor` (`rooms`, `corridor_len`), or `maze` (`braid`); Section 1.4.

- `n_holes` / `target_b1`, `goal_in_chamber`, `seed`; `p_slip` and `reward_mode` are environment-level knobs.

A configuration serializes to the canonical string `TG-GridWorld2D-S{size}-C{c}-D{d}-cs{n}-ds{n}-sep{n}-shp{` which is the run-log key and the manifest row; registry ids are aliases for canonical strings. Custom configurations built through this API pass the same validity checks and ground-truth generation as registry entries, so results on them remain comparable in kind, but only registry entries belong to the published benchmark.

1.4 Environment Modes

- `open`: chambers and decoys placed in a free field by seeded rejection sampling subject to `min_sep`. The direct realization of the theory's objects: dilution scales with `size`, discrimination with `decoy_count`, parallel entry requires the separation condition. Generation: sample footprints uniformly, reject on overlap or separation violation, carve walls and doors, validate.
- `nested`: `depth` concentric shells around one innermost chamber, one door per shell, doors angularly offset so entry forces traversing shells in order. The sequential regime where the parallel-chamber separation deliberately fails. Bars are born sequentially: entering shell j first exposes shell $j{+}1$'s cycle. Validity: shell spacing ≥ 2; depth $\times$ spacing bounded by half the side.
- `corridor`: rooms joined by width-1 corridors of parameterized length, the room graph constrained to a tree. The tree constraint is load-bearing: a cyclic room graph would enclose wall blocks and manufacture decoy-like H_1 bars, while a tree keeps free space simply connected — zero environmental homology signal, only transient exploration pockets. The bottleneck regime; corridor length is the bottleneck-depth difficulty axis. Ground truth additionally exposes the room graph and corridor cells.
- `maze`: seeded perfect-maze carving (a uniform spanning tree over cells), the maximal-corridor-density cousin of `corridor` mode — simply connected at `braid` $= 0$. The `braid` parameter opens loops with the given density; each opened loop encloses wall and therefore creates an incidental H_1 class, so braided mazes interpolate between the bottleneck regime and the discrimination regime.

1.5 Validity checking and manifests

The generator validates before building and rejects with a reason: packing feasibility (footprint plus separations fits, by a conservative area bound and then detection of rejection-sampling failure); exterior 4-connectivity after carving; well-composedness; door width; separation claims. The manifest tool emits a CSV of (registry id, canonical configuration, validity, satisfied assumptions) covering the registry — and, for power users, any custom configuration set. The published benchmark is the registry manifest plus fixed seed lists — tuning seeds and evaluation seeds, identical environments, disjoint seeds.

1.6 Ground truth, certification, and instrumentation

Read-only accessors per instance: the wall mask; the chamber list with interiors, perimeters, and door cells; the decoy list; per-cell annotations (chamber and door membership); and exact snapshot save/restore. The certified metadata attached to every instance records the free space's homology, computed from the selected complex backend (Section 1.8) and cross-checked against the analytic

expectation for the placed features at generation time. Registration-time unit tests assert the standing conventions on every generated instance.

The two door conventions. Every instance certifies its Betti numbers under both readings of a door:

- `betti_z2` (*doors walkable*, the headline): an enclosure with a door is an enterable room, not a hole — its wall is filled before computing, so b_1 counts only *sealed* structure: decoys, doorless chambers, and solid obstacles. A `ChamberCount` world certifies $[1, 0, 0]$; `Decoys4` certifies $[1, 4, 0]$ (the four sealed decoys and nothing else); the `Top` surfaces, whose chambers all have doors, recover the full closed surface (the torus certifies $[1, 2, 1]$, with $H_1 = \mathbb{Z}^2$ and Klein's torsion $\mathbb{Z} \oplus \mathbb{Z}/2$ intact).
- `betti_z2_sealed` (*doors count as walls*): every door is sealed, so each doored interior becomes its own component and every wall component — open arcs included — contributes a class. The raw free-space b_1 (the strict homotopy type of the traversable region, where even a C-shaped wall arc carries a class) coincides with `sealed`[1], so no information is lost between the two readings.

Both are certified against independent analytic expectations at generation time, and the accessors `betti_numbers_doors_dont_count_as_walls()` / `betti_numbers_doors_count_as_walls()` name them unambiguously.

1.7 The variation contract

A benchmark has to say precisely what changes when the seed changes. Every quantity falls into exactly one tier:

- **Frozen by the configuration** — world size, chamber and decoy counts, room shape and side, door convention, corridor length, nesting depth. These *are* the family; no seed touches them.
- **Fixed by the placement policy** — the macro arrangement: a centered chamber, chambers spaced evenly around the perimeter, decoys ringed about the center, the start in the bottom-left corner. This is the family's visual identity and the reason `ChamberCount8` reads as eight chambers rather than eight random blobs; freezing it also means the family's difficulty axis is its parameter, not packing luck.
- **Sampled per seed** — which wall side each door occupies, the goal cell within its chamber, shape assignment under `mixed`, maze and braid structure, corridor routing, hole shapes, and — when `placement_jitter` is positive — a bounded perturbation of each policy anchor.
- **Independent of the seed** — `p_slip`, `reward_mode`, `actions`, `obs_mode`, `complex`, `max_steps`.

Seed **0** is the canonical specimen: the layout the documentation pictures, the manifest certifies, and `gym.make` returns when no seed is given. It is canonical only by being the default — structurally it obeys the same contract as every other seed, because a benchmark whose first sample differs in kind from the rest is a trap for anyone building splits. Procedural mode, where the layout is resampled every episode, is available explicitly rather than as the accident of omitting a seed.

Setting `placement="random"` drops the macro arrangement into the sampled tier wholesale. That is the honest switch for anyone who wants unconstrained placement; jitter is the gentler one, and the benchmark splits use it (Section 6).

1.8 Complex backends (all variants)

Every environment exposes a `complex` configuration field selecting the geometric substrate used for free-space homology:

- `cubical` (default) — the glued cubical cell complex of the free cells; homology is computed by GUDHI [2] on the order complex of its face poset (the barycentric subdivision), which is robust to every edge identification of Section 3. This backend is the certification source of truth: the metadata's Betti numbers are computed here and cross-checked against the analytic expectation at generation time.
- `rips` — a Vietoris–Rips complex built by GUDHI on the centers of the free cells at a fixed scale $t \in (\sqrt{2}, 2)$, under the quotient metric induced by the base's identification group (wrap translates and flip glide reflections). Orthogonal and diagonal neighbors connect, cells separated by a width-one wall (ambient distance 2) do not, so obstacles produce classes exactly as in the cubical complex; the backend reports dimensions 0 and 1, and the library's tests assert agreement with the cubical computation of the raw free space on every base (the strict traversable-space reading; its b_1 equals `sealed`[1]).

The backend governs the environment-side homology computations (`free_betti`, `visited_betti`, `observed_betti`) and is recorded in the metadata; certification always comes from the cubical complex.

Determinism. The library guarantees end-to-end determinism up to seeds: (configuration, seed) fixes the generated layout and its certified metadata byte-for-byte — including everything computed through GUDHI, since homology is exact linear algebra over $\mathbb{F}_p$ with no randomness — and (environment, reset seed, action sequence) fixes the episode, including `p_slip` resampling, which draws from the episode's seeded generator. Internal iteration orders are sorted so no result depends on interpreter hash state; a cross-process test asserts that two fresh interpreters produce identical layouts, metadata, and rollouts.

1.9 Reward modes (all variants)

Every benchmark variant (GridWorld2D, -Texture, -Top) supports a `reward_mode` configuration field. Episodes have a pre-determined length (Section 1.10), fixed by (configuration, seed) before the episode starts and overridable via `max_steps` — and every environment places a goal cell by default (removable with `goal=False` for goal-free exploration studies):

- `sparse` (default) — +1 terminal reward on reaching the goal cell, placed inside a designated chamber (in Texture scenarios, the treasure-chest cell). Placement inside a chamber makes steps-to-first-reward coincide with steps-to-first-entry, so the reward metric stays continuous with the exploration metric.
- `none` — no extrinsic reward; the environment is a pure-exploration benchmark, and event-based metrics (entry times, coverage) carry the evaluation.
- `coverage` — +1 for each cell visited for the first time: a dense pure-exploration reward for methods that require a reward signal.
- `deceptive` — the `sparse` target plus a distractor shaping gradient leading away from the target's door: the reward-deception regime of the novelty-search literature, letting reward deception be measured rather than only structural deception.

Ground truth exposes the reward cells and (for `deceptive`) the shaping field. Reward modes exist for community use, for training reward-consuming baselines, and for deception studies; pure exploration evaluations should run `none` to keep an extra variable out of their results.

1.10 Episode horizons

The horizon must make the goal *reachable with room to spare*. It is not an exploration budget for a single episode: agents learn across many episodes, exploring early and exploiting later, so one episode never has to cover the world — it has to permit reaching the goal and wandering on the way.

$$\text{horizon} \;=\; \max\!\big(\lfloor 1.2\max(W,H)\rfloor,\; 10\,\big\lceil \tfrac{k\cdot d^\star}{10}\big\rceil\big)\,, \qquad k=3,$$

where $d^\star$ is the *turn-aware* optimal: the fewest actions from start to goal counted in the egocentric action space, so the quarter-turns a corridor forces are charged (a breadth- first search over cell $\times$ facing). It is computed once per (configuration, seed), cached on the layout, and reported in the manifest, which is what makes a split's difficulty distribution auditable rather than assumed. $d^\star$ is measured in the egocentric space regardless of the configured action mode: the horizon is a property of the world, so selecting `actions="fourway"` must not silently shorten it.

The size floor preserves the short-rollout intent (60 on a 50-grid, 120 on a 100-grid) wherever it is already generous, and the second term lifts it only where the structure demands. That second term is not a refinement: shortest paths in mazes, corridor trees, and nested shells grow far faster than the side length — a 50×50 perfect maze needs 674 actions — so a size-only rule leaves those families with goals no agent, however omniscient, could reach inside the episode.

Worst-case dynamics. Where the world changes during an episode, $d^\star$ is computed against the most constrained configuration the schedule can produce — for `EnvironmentalIceShip`, every cell any freeze wave can close. Ice only ever shrinks from that maximum, so a route valid there is valid at every step, and *every* season draw is solvable rather than only the fortunate one. Teleporters are honoured too: a route through a wormhole is a real route, which is how `SpaceWarp`'s treasure chamber — spatially its own component by construction — is reachable at all.

1.11 Universal action and observation spaces

Both action modes are identical across all variants (GridWorld2D, -Texture, -Top); an agent written against either runs on every environment in the library unchanged.

Action space. The default is *egocentric* `Discrete(3)`: turn left, turn right, step forward. The agent carries a parallel-transported frame, so a flipped seam (Möbius, Klein, $\mathbb{RP}^2$) genuinely mirrors its view — and its heading is always visible in the rendering (a MiniGrid-style arrow, or the scenario sprite, rotated to face forward). Moving into an obstacle leaves the agent in place; with `p_slip` > 0 the executed action is resampled uniformly with that probability.

The override `actions="fourway"` selects the spec's universal `Discrete(4)` space: up, down, left, right, always denoting *screen* directions (up decreases y) — the frame is re-canonicalized every step, so after any seam crossing left is still screen-left, and only the *position* feels the identification map. Family-specific cell mechanics (hazard cells that reset the episode, wormhole cells that teleport; Section 2) are documented with the family and never alter either action set.

Observation space. The default observation follows the action mode: egocentric agents receive an occluded egocentric window — a $(2r{+}1) \times (2r{+}1)$ symbolic patch (default $r = 3$), agent centered and facing up, walls occluding line of sight. The spec's *universal vector* observation (default under `fourway`, available everywhere via `obs_mode="vector"`) is the agent's integer cell coordinates (x, y) followed by a texture block $t \in [0, 1]^{16}$: slots 0–3 are reserved library-wide for directional blocker adjacency (obstacle to the left, right, above, below of the current cell); slots 4–15 carry per-environment semantic features documented with each family. The texture block is identically zero in GridWorld2D and GridWorld2D-Top; only GridWorld2D-Texture populates it. `obs_mode="global"` (the flattened symbolic grid with an agent mask) completes the set; all three are available on every variant. Snapshot save/restore is exact (deep copy), and the environment is deterministic given (configuration, seed, action sequence) when `p_slip` $= 0$.

1.12 Gallery

Figures 1 and 2 render every pinned GridWorld2D id at the reference seed (reveal mode, uniform frame size).

2 GridWorld2D-Texture

The textured variant (`TG-GridWorld2DTex` prefix): the same generator with a per-cell texture channel — semantic labels such as blocker-adjacent (ice), door, ladder, water, and scenario-specific others — exposed in the observation and recorded in ground truth. Texture is a third signal family: local like curvature, but semantic rather than geometric, and only as trustworthy as the environment's labeling.

Design intent. Texture scenarios are built so the taxonomy's boundary is visible: texture discriminates flagged structure cheaply where the labeling is faithful, and fails exactly where discrimination requires enclosure rather than adjacency. The flavor example: arctic sailing — most holes are ice-cube decoys whose neighbors carry ice-adjacent texture (avoidable on sight); the target is a treasure chest in an enclosed space reachable through a water-only passage; texture flags ice adjacency and doorways (ice on both sides), while whether the passage leads anywhere remains invisible to any local feature. Scenario suites should include cells where texture is absent or misleading, so the boundary is measured rather than assumed.

Texture encoding. Textures populate the universal observation's texture block (Section 1.11): slots 0–3 carry directional blocker adjacency (blocker to the left, right, above, below — any combination), and slots 4–15 carry the scenario's semantic features, documented per environment below.

Canonical environments. Eight named environments form the Texture slice of TopoGym-v1:

- `TopoGym/IceShip`. Arctic ship navigation (the agent is the sailboat, and *hitting ice ends the episode* — collisions hurt the hull). Coastal ice sheets attach to the north and south edges — land masses, not floating obstacles — octagonal sealed bergs float in the open water as decoys, and the treasure sits in a cavity deep inside the large eastern landmass, reachable *only* through a guaranteed narrow width-1 channel threaded across the ice (the guarantee is unit-tested: blocking the channel disconnects the goal). Ice-adjacent water cells carry the directional adjacency flags — the boat's collision-warning system — and open-water cells carry

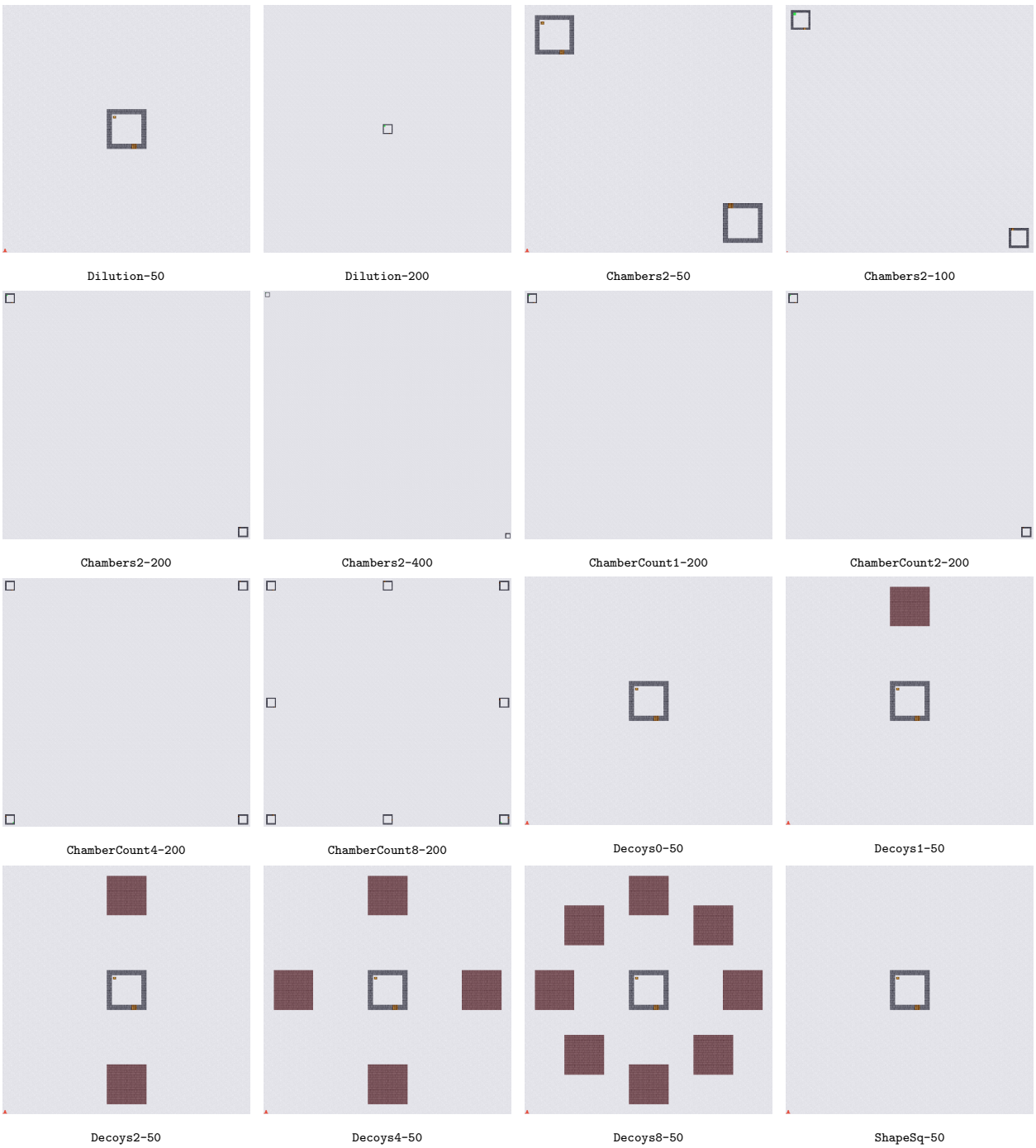

Figure 1: The GridWorld2D registry (1 of 2): reference-seed renders, reveal mode.

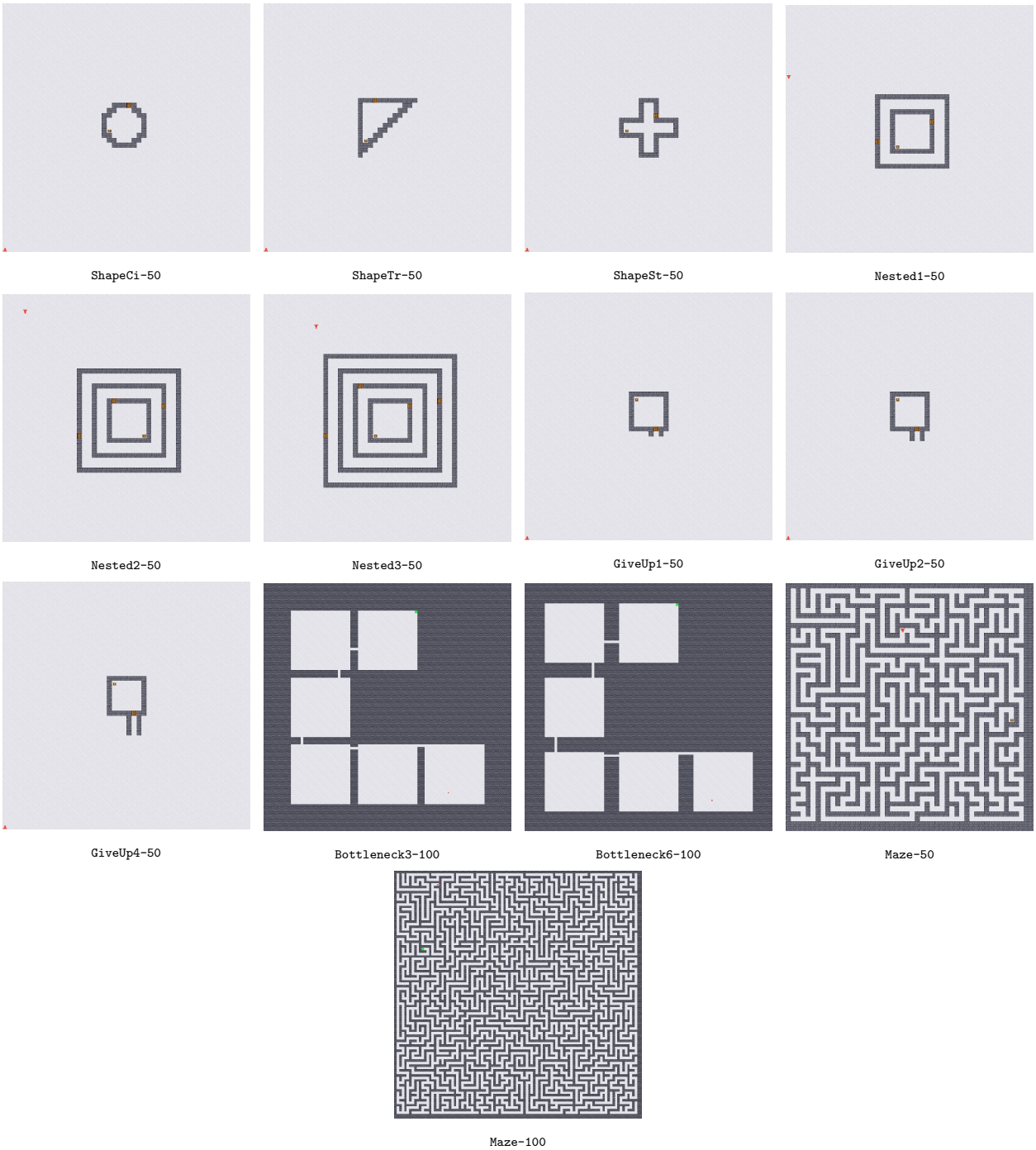

Figure 2: The GridWorld2D registry (2 of 2).

the water feature only. Texture makes berg avoidance cheap on sight; whether the channel leads anywhere remains invisible to any local feature.

- `TopoGym/Ladders`. Platforms, ladders, and bridges; a gem on the top platform is the target. Rooms (platforms), ladders, and bridges each carry their own semantic feature; vertical progress is only possible on ladder cells, and bridges connect platforms across gaps. The textured version of the bottleneck regime: ladders and bridges are the corridors, and their texture advertises them.

- `TopoGym/BankRobber`. Nested rooms with the money in the center room — the textured version of the nested regime. Door cells and hallway cells carry their own features, so the sequential structure is advertised locally; the ordering constraint (which door must come first) remains global.

- `TopoGym/DontFall`. A large world with a large central hole: stepping onto the drop resets the episode (the hazard mechanic). A few dozen much smaller huts surround the hole, and exactly one contains the ruby. Textures distinguish drop-adjacent squares, regular dirt, hut interiors, and hut doorways. The hazard makes local novelty signals actively dangerous — the most novel direction is the drop — while the huts reproduce the discrimination regime at scale.

- `TopoGym/SpaceWarp`. Four chambers and a *field* of wormholes: 30+ paired teleporter cells scattered evenly across the map on a jittered lattice (kwarg `warp_sep` sets the minimum spacing; never on or next to a doorway, so every door remains enterable). Stepping onto a wormhole teleports the agent to its partner (the teleport mechanic). The treasure chamber has no door at all: from outside it is indistinguishable from a sealed decoy, and it is enterable only through one wormhole pair running chamber-interior to chamber-interior. Texture flags a cell as a wormhole but never which one — all wormholes are purple — so wormholes are recognizable on sight while their destinations must be explored. The field is the point: it is constant noise for any gradient-follower, while methods that track local transition structure can model each jump once traversed. Reachability of every chamber under wormhole transitions is guaranteed and unit-tested.

- `TopoGym/ClownChase`. The reward-deception scenario. Three sealed decoy tents cluster on one side of the map with a troupe of clown NPCs (`n_clowns` make-kwarg, default two) random-walking among them, each leashed to its own tent; every agent step that reduces the distance to the *nearest* clown pays a tiny reward (10^{-3}) from one shared budget of 2.0 that spans the agent's lifetime on the layout — the distractors run out after a few thousand rewarding steps in total. The treasure chamber sits on the opposite side (the construction rejects layouts where the sides are not genuinely opposite). Texture distinguishes doors, floor, room interiors, the treasure cell, and clown proximity (a slot lights up when any clown is within one cell); the clowns are otherwise sensed only through the reward gradient — which is exactly the signal that must be distrusted.

- `TopoGym/EnvironmentalIceShip`. IceShip with seasons and a structured landmass: three cavities — one holding the treasure, two empty — each behind its own guaranteed channel, plus two sealed water pockets (little enclosed lakes, visible but unreachable: the certified $b_0 = 3$ counts them). Each episode is drawn sunny (summer) or snowing (winter), with matching palettes; the water texture's value encodes the season (1.0 sunny, 0.5 cold). Only the *floating bergs* respond to the season: in winter they grow — their water fringe freezes in waves at steps 20 and 40, and being on a cell when it freezes ends the episode — and in summer their rims

melt away. Channels and the landmass never change. The certified metadata describes the episode-start geometry; a `season` make-kwarg forces either season for controlled evaluation.

- `TopoGym/SearchRescue`. Search and rescue in a collapsed concrete structure. A person is trapped in the one large intact chamber; around it, 160 rubble blocks on a dense lattice leave only width-1/2 passages — a maze the rescuer must thread, with the start placed far from the chamber. Every block is a small H_1 class of its own (certified $b_1 = 161$); in the agent's own discovery filtration — the archive — rubble loops are born small and die quickly as their blocks are circumnavigated, while the chamber's enclosing class grows large and refuses to die. Persistence, not luck, finds the person. Ten explosive red barrels sit in the passages (flame-marked, fatal to step on; neighbors carry the hazard-adjacent texture); a barrel-free route to the person is guaranteed.

SpaceWarp and local evidence (semantics, not soundness). SpaceWarp is built to separate two statements that coincide everywhere else in the library: "this enclosure admits no local entry" and "this enclosure cannot be entered." The treasure chamber's enclosing class is genuinely permanent in the spatial complex (its wall has no door, so the hugging cycle never bounds even after the interior has been visited), and exhaustive local probing of its wall truthfully reports no entry — yet the chamber is enterable, through a non-local transition. Methods that classify enclosures from local evidence alone will mark it unenterable; methods that account for non-local transitions — e.g. by analyzing the transition graph of experience, where a wormhole edge appears once traversed — can do better. SpaceWarp makes that semantic boundary measurable.

Semantic slot assignments. Slots are assigned library-wide so an agent transfers between scenarios; each scenario uses the subset relevant to it. Cell mechanics (hazard, wormhole) are distinct cell types, visible in every observation mode.

slot	feature	slot	feature
0–3	blocker adjacency (L/R/up/down)	10	drop-adjacent
4	water	11	plain ground
5	platform	12	room interior
6	ladder (vertical corridor)	13	standing on a wormhole
7	bridge (horizontal corridor)	14	clown within one cell
8	doorway	15	standing on the treasure
9	hallway		

Scenario sizes are fixed (IceShip/EnvironmentalIceShip/Ladders/BankRobber/SpaceWarp: 50; DontFall/SearchRescue: 61; ClownChase: 60); layouts vary with the seed under each scenario's structural guarantees.

2.1 Gallery

Figure 3 renders every Texture scenario at the reference seed.

3 GridWorld2D-Top

The topological variant (`TG-GridWorld2DTop` prefix): non-trivial global topology via edge identifications of the grid's boundary. The `topology` field selects the identification:

Value	Identification	Space	$H_1(\cdot;\mathbb{Z})$	$H_1(\cdot;\mathbb{F}_2)$
`plane`	none (walled boundary)	disk	0	0
`cylinder`	left–right, straight	cylinder	$\mathbb{Z}$	$\mathbb{F}_2$
`mobius`	left–right, flipped	Möbius band	$\mathbb{Z}$	$\mathbb{F}_2$
`torus`	both pairs, straight	torus	$\mathbb{Z}^2$	$\mathbb{F}_2^2$
`klein`	one straight, one flipped	Klein bottle	$\mathbb{Z} \oplus \mathbb{Z}/2$	$\mathbb{F}_2^2$
`rp2`	both pairs, flipped	real projective plane	$\mathbb{Z}/2$	$\mathbb{F}_2$

Movement wraps across identified edges, with coordinate reversal where the identification is flipped; obstacles and chambers can be placed on top of any topology. Note the torsion: over $\mathbb{Z}$ the Klein bottle and $\mathbb{RP}^2$ carry $\mathbb{Z}/2$ classes that integer rank alone misses, while $\mathbb{F}_2$ coefficients see them — one reason the library computes homology over $\mathbb{F}_2$.

Why it matters. These spaces are locally flat everywhere: no local signal — curvature, texture, novelty gradient, count statistics — distinguishes a Möbius band from a plane at any single cell, because every neighborhood is Euclidean. The global structure is visible only to signals that aggregate globally, and the ambient H_1 is non-trivial with no obstacles at all. This is the regime designed to break methods that rely on local exploration gradients, and the cleanest possible separation between local and global signal families.

Canonical layout. The Top environments share one scenario shape: chambers placed near the corners of the large fundamental square, exactly one holding the treasure. The square's sides are the edges of the identification diagram (the fundamental polygon), so the corner regions meet across the identified edges — on the torus all four corners are a single point of the quotient; on the Klein bottle and $\mathbb{RP}^2$ they meet in pairs with orientation reversal. What appears locally as four maximally separated chambers is globally a tight cluster reachable across the edges; an agent that carries only local structure treats the corners as far apart, while the quotient makes them neighbors.

Registry entries. The canonical layout is registered per topology as `TopoGym/Top{Plane, Cylinder, Mobius, Torus, Klein, RP2}-50-v0`: four chambers of side 8 near the corners of the fundamental square, exactly one holding the treasure, start near the center. Certified b_1 equals the ambient rank plus the four chamber classes (plane and $\mathbb{RP}^2$: 4; cylinder, Möbius, torus, Klein: 5). The identification-aware Rips backend (Section 1.8) computes homology on the quotient metric and is tested to agree with the cubical certification on every topology. Detection protocols separating the ambient classes from feature classes are left to evaluation code; the metadata provides both the ambient base facts and the composed totals.

Reading the seam markers. Every rendered frame draws the fundamental-polygon notation on identified edges, so the identification *and its orientation* are visible in the environment itself (and in Figure 4): chevrons run along each identified edge pair, one color per pair — yellow for the left/right (x) pair, cyan for the top/bottom (y) pair. Chevrons pointing the *same* direction on both edges mean the pair is glued straight (a wrap, as on the cylinder and torus); chevrons pointing *opposite* directions mean the gluing reverses the edge (a flip, as on the Möbius x-pair, the Klein y-pair, and both pairs of $\mathbb{RP}^2$). Unmarked boundary edges are walls. This is exactly the arrow convention of the standard fundamental polygons ($aba^{-1}b^{-1}$ for the torus, $abab^{-1}$ for the Klein bottle, $abab$ for $\mathbb{RP}^2$), drawn in place on the grid. The four corner chambers are anchored symmetrically — chambers in the same row pair share identical row extents and same-column pairs

share identical column extents — so any apparent asymmetry in a render is the identification, never the layout.

3.1 Gallery

Figure 4 renders the six Top environments; the colored seam markers show which edge pairs are identified and with what orientation.

4 The library: code interfaces

TopoGym is a Gymnasium library (`pip install topogym`; MIT). This section documents the key abstractions with usage examples; the repository's `docs/` tree carries per-environment pages and the internals reference.

4.1 The registry (the primary interface)

Importing `topogym` registers every TopoGym-v1 id. The seed selects the layout within the frozen configuration; documented knobs pass through `gym.make`:

```
import gymnasium as gym
import topogym  # registers the TopoGym/* ids

env = gym.make("TopoGym/Decoys4-50-v0", seed=3, p_slip=0.05)
obs, info = env.reset(seed=0)
info["topology"]["betti_z2"]         # [1, 4, 0]  (doors walkable)
info["topology"]["betti_z2_sealed"]  # [2, 5, 0]  (doors count as walls)

from topogym import registry
registry.registry_ids()              # every pinned id
registry.canonical_string(
    registry.get_config("TopoGym/Decoys4-50-v0"), seed=3)
# 'TG-GridWorld2D-S50-C1-D4-cs8-ds8-sep2-shpSq-open-slip0-seed3'
registry.manifest(seed=0)            # validity + assumptions per entry
```

Episodes truncate after the pre-determined horizon (Section 1.10); the sparse goal pays +1 by default (Section 1.9). Archive-style teleport resets are opt-in:

```
env = gym.make("TopoGym/Maze-100-v0", seed=1, teleport=True)
env.reset(seed=0)
# ... explore ...
env.reset(options={"teleport": (12, 40)})  # any visited cell
```

4.2 The generator and the fluent spec API

Custom configurations pass the same validity checks and certification as registry entries:

```
from topogym.generation import TopoGenConfig2D, generate_2d

cfg = TopoGenConfig2D(
    base="klein", size=25, n_chambers=1, n_decoys=2, n_holes=0,
    chamber_shape="circle", chamber_side=8, min_sep=3,
    door_kind="open", door_corridor_len=2,
```

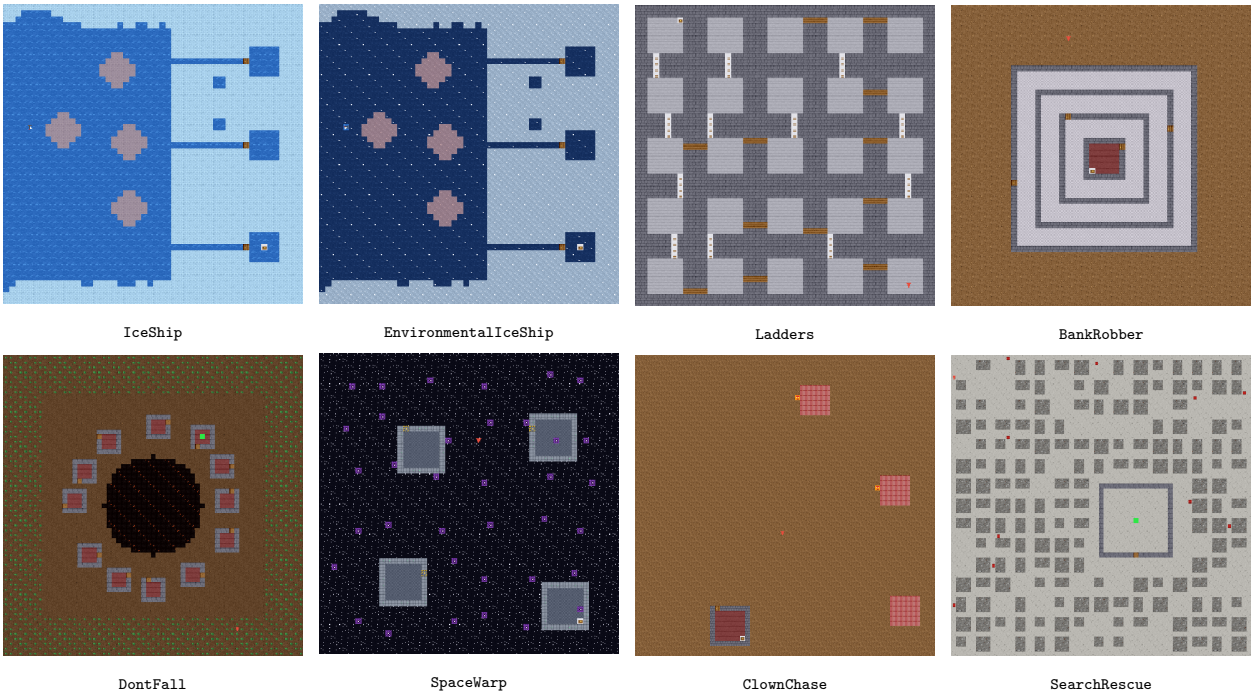

Figure 3: The Texture scenarios: reference-seed renders, reveal mode.

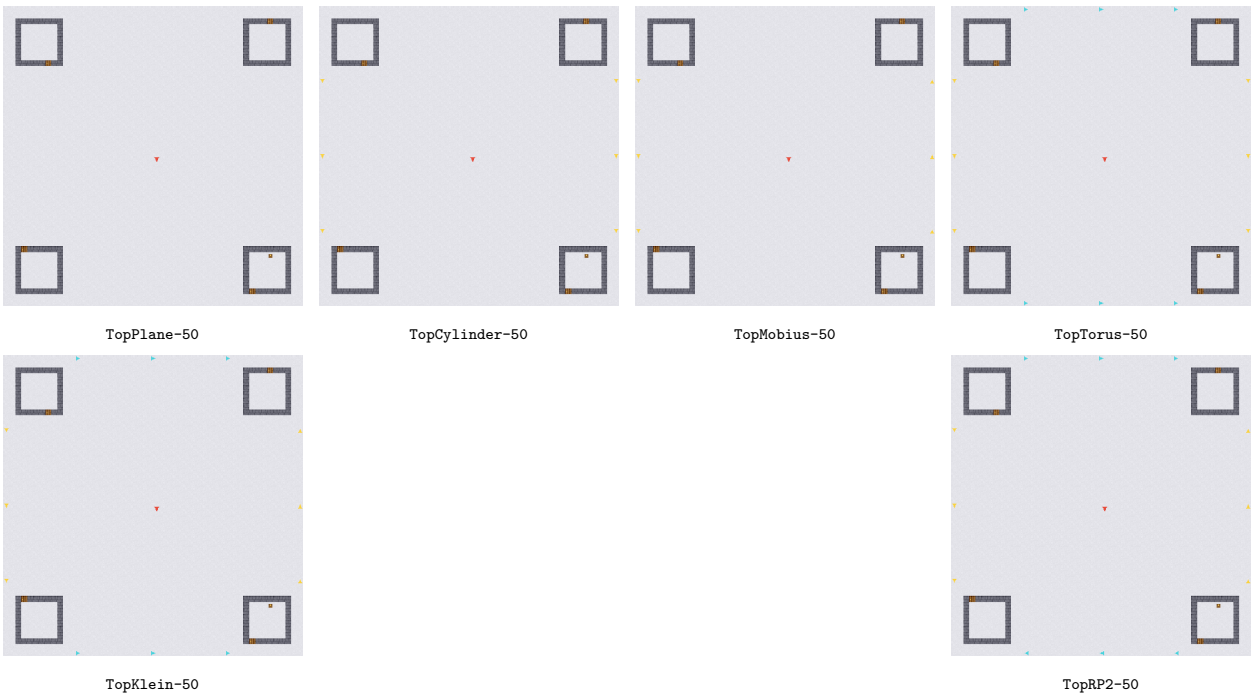

Figure 4: The Top slice: corner chambers on each identification surface (edge identifications marked on the seams).

```
)
layout = generate_2d(cfg, seed=42)  # deterministic + certified
layout.metadata.betti_z2

from topogym.spec import Torus
env = Torus(15).holes(3).chambers(1).compile(seed=7)
```

4.3 Measuring discovery

`ExplorationTracker` turns a rollout into discovery-time persistence (essential bars are the real topology of the explored region; finite bars are transient beliefs), and `StatsRecorder` accumulates per-episode metric rows:

```
from topogym.stats import StatsRecorder
from topogym.tda import ExplorationTracker

env = StatsRecorder(gym.make("TopoGym/Nested3-50-v0", seed=1))
tracker = ExplorationTracker(env)
tracker.reset(seed=0)
# ... run the policy ...
tracker.summary()      # coverage, recovery steps, bar counts
env.episodes           # return, coverage, chamber entry steps, ...
m = env.metrics()      # the standardized metric set (frozen dataclass)
m.success_rate; m.sample_efficiency; m.visitation_entropy
m.mean_regret; m.planning_efficiency   # vs env.shortest_path()
m.steps_to_coverage    # {0.5: step, ..., 1.0: step}
m.steps_to_h1_holes    # {k: step the k-th loop was discovered}
```

The metrics interface and its toggles. Metrics that are potentially expensive are *off by default* and enabled per recorder; everything cheap is always on. The toggles:

- `record_steps` — keep a per-step row (reward, coverage, lifetime coverage) enabling `coverage_at(global_step` cheap but memory-linear in steps.
- `track_holes` — recompute the observed region's homology every step to timestamp discoveries: `steps_to_h0_holes` and `steps_to_h1_holes` map k to the global step at which the observed region first carried $h_i \geq k$ (runs GUDHI per step).
- `track_curvature` — include `curvature_coverage_below_zero`: the fraction of negatively Ollivier–Ricci-curved cells (corridors, doorways, bottlenecks) the agent has reached. Curvature is exact ($\alpha = 0$, unit-expanded W_1 via Hungarian matching), computed once per layout and cached; `recorder.curvature_coverage(x)` answers "percent of cells with curvature $< x$ reached" for any threshold regardless of the toggle, and `env.ollivier_ricci()` exposes the raw per-cell field.

```
env = StatsRecorder(gym.make("TopoGym/Decoys4-50-v0", seed=1),
                    record_steps=True, track_holes=True,
                    track_curvature=True)
# ... run episodes ...
m = env.metrics()          # frozen dataclass; m.to_dict() for logging
m.steps_to_h1_holes        # {k: global step the k-th loop was found}
```

```
m.curvature_coverage_below_zero
env.curvature_coverage(-0.2)    # any threshold, on demand
env.unwrapped.homology_stats("observed")  # HomologyStats(h0, h1, h2=None)
```

`env.homology_stats(which)` returns per-dimension hole counts as a `HomologyStats` value object for `"observed"`, `"visited"`, `"certified"`, or `"certified_sealed"`; gridworlds carry h_0 and h_1, with h_2/h_3 optional (`None` where the dimension does not apply).

Standardized logging. All library logging flows through the `topogym` logger (with a `NullHandler` installed, so it is silent until the host configures `logging`): episode summaries at INFO, the `TOPOGYM_DEBUG` stream at DEBUG. `recorder.save(path)` writes the standardized run log as JSON — a header carrying the canonical run key (`recorder.run_key()`, the configuration serialized to the canonical string of Section 1.3), the library version, and the certified topology; then the episode rows, the metric set, and optionally the step rows. The log is a pure function of the run — no wall-clock timestamps — so replaying a seed reproduces it byte for byte.

Structure accessors ground the metrics: `env.topology.betti_numbers_doors_count_as_walls()` / `_dont_count_as_walls()` return a `BettiNumbers` value object; `env.graph()` exposes the free-cell graph as a `networkx.Graph`; `env.shortest_path()` the optimal baseline; `env.bottlenecks()` the straight-through width-1 passage cells (logged at reset under `TOPOGYM_DEBUG`).

Per-step `info` carries `coverage`, `lifetime_coverage` (across episodes on a layout), `chambers_entered`, `episode_return`, `h0_merges`, and friends; `env.unwrapped.chamber_entry_steps` gives each chamber's first-entry step.

4.4 Learning from topology: the `VisitedComplex`

The `VisitedComplex` is the library's interface for building custom topological RL algorithms: an incrementally maintained cell complex over the states an agent has *visited* — the archive-restorable set — from which agents read Betti numbers, torsion, representative cycles, and rims, using their own encoders and their own choice of complex. It is deliberately independent of the environment's certification pipeline: the backend, scale, and coefficient ring can all differ from what the env itself uses.

```
from topogym.tda import VisitedComplex

vc = VisitedComplex.from_env(env)      # seeded with lifetime visits
vc.add(new_cells)                      # feed states as you explore
vc.betti()                             # (b0, b1) over the chosen ring
vc.representatives()                   # one closed loop of cells per
                                       #   H1 generator: teleportable
vc.rims(observed=seen)                 # where each loop can tighten
vc.torsion(1)                          # integral torsion of H1
```

Backends. Three complexes, each with its parameters:

- `cubical` (cell environments; takes the env's glued base map): vertices are the visited cells, edges their 4-adjacencies (seam gluings included), and a square fills wherever the full star of a base corner is visited. Representative cycles are therefore loops *of cells* — ordered sequences of archive targets — and diagonally touching trails stay disconnected, matching the movement convention. A fully-visited `TopKlein` recovers the Klein bottle exactly (below).

- `vr` (`epsilon`, default 1.5): Vietoris–Rips on arbitrary points — visited cells under the base's quotient metric, or any encoder's vectors via `metric=`. Cliques are built to `max_dim+1`, so `max_dim=2` computes H_2 (a sampled sphere reports $(1, 0, 1)$).
- `witness` (`landmark_radius`, `relaxation`): the landmark witness complex of de Silva and Carlsson [3]. Every visited point witnesses; a sparse landmark subset spans the complex, so it stays small on long trajectories. Landmark selection is a *policy you override*: `landmark_policy(point, landmarks, dist)` → `(admit, evict)` decides whether a new state becomes a landmark and which existing one (if any) is kicked out — the default admits maxmin-style at `landmark_radius`, and a six-slot ring buffer is three lines.

Coefficients. `coefficients` takes any prime ($\mathbb{F}_2$ default, $\mathbb{F}_3$, ...; Gaussian elimination mod p) or `"Z"` (Smith normal form), where `torsion()` becomes meaningful. The standard example is visible in-library: a fully-visited Klein bottle reports $b_1 = 2$ over $\mathbb{F}_2$, $b_1 = 1$ over $\mathbb{F}_3$, and $H_1 = \mathbb{Z} \oplus \mathbb{Z}/2$ integrally [4].

Recommended use. This is the intended way to consume TopoGym's topology in an agent. Rather than reading ground truth out of the environment, an agent feeds the structure what it has actually visited and reads back the shape of what it knows — a map of the holes it has found and the loops that enclose them, in its own choice of complex and coefficient ring. The representative cycles are the practical payload: each is a closed walk through archive-restorable states, so an agent can treat them as targets to return to, edges to push outward, or features to encode. Think of it as the agent's own map to the interesting holes in a world, learned from experience and usable as a signal — while the certified metadata stays where it belongs, as the answer key for scoring rather than an input.

Cost. Computation is lazy and cached but *not* incremental: `add` records points and invalidates, and the next query rebuilds. Measured on the cubical backend over a dense visited region:

cells	build	betti	representatives	rims
1,000	0.02 s	0.01 s	0.02 s	∼0
10,000	0.26 s	0.24 s	0.73 s	∼0
50,000	1.40 s	1.57 s	12.7 s	∼0
100,000	3.04 s	5.55 s	43.6 s	0.01 s

The build and rims are linear in the archive, `betti` near-linear, and `representatives` superlinear — comfortable to roughly 20,000 cells and expensive past 50,000. The costs are sequential, so extracting cycles from a 100,000-cell archive is the build plus the extraction, about 47 s in total. For scale, only `Chambers2-400` has more free cells than that; a 50-grid archive is around 2,500 cells, where the same call takes 0.02 s. Query once an episode rather than once a step, and treat `torsion` — an integer Smith normal form, minutes at 20,000 cells — as an offline diagnostic.

Design. All three backends reduce to one `ChainComplex` (cells per dimension, integer boundary maps), so higher-dimensional features arrive by adding cells, not new machinery; `add()` is cheap and rebuilds lazily; every query is deterministic (sorted iteration); progress logs flow through the `topogym` logger at DEBUG. Representative loops are fundamental cycles of a spanning forest kept when independent modulo the 2-cell boundary space [5], so each returned generator is an actual closed walk through visited states — the same objects the H1 overlay draws and `h1_representatives()` returns, generalized to user-selected complexes.

4.5 Playing and rendering

Every environment renders as `rgb_array` (procedural pixel-art tiles), `ansi`, or `human` (a pygame window; `pip install topogym[play]`). The keyboard driver:

```
python scripts/play.py --list
python scripts/play.py TopoGym/SpaceWarp-v0 --seed 2
# arrows move; tab reveals hidden structure; r resets;
# backspace regenerates the layout with the next seed
```

Rendering always dims cells outside the agent's current line of sight, so what the agent can and cannot see is visible at a glance (reveal mode shows the full map undimmed; it exists for documentation).

4.6 Debugging the topology live

Three environment variables turn the simulator into a topology debugger; all are off by default and cost nothing when unset.

The H1 overlay. `TOPOGYM_OVERLAY=1` (alias `OVERLAY_ENABLED=1`) recomputes, on every rendered frame, the H_1 classes of the *strictly-visited region* — the cells the agent has actually stood on, which are exactly the states an archive can restore to — and draws each class on the grid:

- the **representative cycle** (yellow): the innermost closed loop through strictly-visited cells enclosing the pocket. Every cell on it is a valid archive/teleport target; cells merely *seen* never appear. The loop is loose when the agent has only encircled from afar and tightens as its trail hugs the structure;
- the **rim** (green): the part of the cycle adjacent to seen-but-unvisited free space — where the loop can still tighten. A class whose rim has gone dark is as tight as the walls allow; a class that dies when its pocket is walked was a transient belief, not a hole. Watching cycles tighten and transient ones collapse is the fastest way to build intuition for the discovery filtration of Section 1.

A legend with the live H_1 count sits in the top-right corner. `env.h1_representatives()` returns exactly what is drawn (one {`cycle, rim, pocket`} record per class, every cycle cell strictly visited), so archive methods consume the same loops the overlay shows — what you see is the interface.

```
# watch loops be conjectured, confirmed, and refuted as you drive:
TOPOGYM_OVERLAY=1 python scripts/play.py TopoGym/Decoys4-50-v0
```

The Ollivier–Ricci heatmap. `OLLIVIER_HEATMAP=1` tints every free cell by its Ollivier–Ricci curvature (the per-cell field of `env.ollivier_ricci()`, computed once and cached per layout): the most negatively curved cells — doorways, corridors, bottlenecks — glow strongest red, fading to untinted at the layout's maximum curvature. A legend in the top-left corner shows the gradient bar with the scale's minimum and maximum values, so the colors read as numbers. Driving through a world with the heatmap on makes the geometric signature of its bottlenecks visible at a glance — the red ridges are exactly the cells the curvature-coverage metric of Section 4 tracks [6]. It composes with the other two variables.

The step stream. `TOPOGYM_DEBUG=1` streams everything the environment computes to the console: a reset line (layout, seed, certified Betti numbers in both door conventions, the surface invariants — Euler characteristic, orientability, genus or demigenus, boundary components — horizon, reward mode, and the bottleneck cells) and one line per step with position, action, reward, running return, termination flags, coverage, lifetime coverage, observed fraction, known components, H0 merges, the observed region's live Euler characteristic, bottlenecks seen so far, chamber entries, doors opened, live scenario state (season and frozen/melted counts, clown positions and remaining budget, hazard counts), and the observation itself rendered human-readably (named texture slots in vector mode; the occluded egocentric patch as ASCII in local mode). The variables compose:

```
TOPOGYM_DEBUG=1 TOPOGYM_OVERLAY=1 \
    python scripts/play.py TopoGym/SearchRescue-v0
```

5 Performance Considerations

The library keeps certification exact while making the training loop cheap; all figures below were measured on one workstation (single-threaded CPython) and matter as ratios, not absolutes.

The step path does no topology. Homology never runs inside `step()`: certification happens once at generation, and every potentially expensive statistic (per-step recording, hole timestamps, curvature) is an opt-in toggle (Section 4). What remains per step is movement, reward, and sight.

Sight memoization. Line-of-sight is the step path's only real cost — an occlusion flood over the egocentric window. Layouts are frozen, so the flood is a pure function of the agent state plus a small *sight token*: everything that can change how a cell looks (opened hidden doors; in seasonal scenarios the season and its freeze/melt progress). Patches are memoized per layout keyed on (state, token) — turning in place and revisiting cells become dictionary lookups, and the token invalidates exactly when appearance changes. The observed-region bookkeeping is replayed on every hit, so discovery accounting (and therefore every metric) is identical with and without the cache; a test pins a cached rollout byte-for-byte against a cold one. Measured effect: the default environment goes from ∼6k to ∼220k steps/s (37×); Texture scenarios reach ∼130k.

Layout cache. Generation is deterministic, so a (configuration, seed) key names exactly one certified layout; building it twice is pure waste. A process-wide bounded LRU stores the first build and hands every later construction an independent copy (immutable base map and metadata shared; mutable containers copied, so instances cannot leak state into one another). `gym.make` + `reset` of a cached id drops from ∼190 ms to ∼0.6 ms — which is what makes vector environments cheap to build.

Vector environments. The standard Gymnasium scaling path works unchanged:

```
venv = gymnasium.make_vec("TopoGym/Decoys4-50-v0", num_envs=8,
                          vectorization_mode="sync", seed=1)
```

With the caches in place the per-copy construction cost is negligible and the base environment is fast enough that the vector wrapper itself dominates; for throughput beyond a few hundred thousand steps/s, batch at the experiment level (e.g. Ray via `topogym[benchmarks]`) rather than expecting more from a single process.

Determinism is unaffected. Both caches are keyed on exactly the state that generation and sight depend on, and cached results are replayed, not approximated — the end-to-end determinism guarantee (Section 1.8) holds with caches warm or cold, and the cross-process determinism test runs with them enabled.

6 Benchmarks

(Outline — the benchmark suite and its numbers land in a future revision; the split manifests will be generated and pinned exactly like the registry manifest of Section 1.2.)

6.1 The comparative benchmark

Splits are drawn over the product of (family, size, seed), not over registry ids: the registry is a curated showcase of canonical specimens, while a split instance is a *sample from a family distribution*. Configurations already serialize to canonical strings, so a split row is a canonical string and needs no new id — which is what lets every family appear at two or more sizes in every split without the registry growing to match.

Seeds come from disjoint, widely separated bands, so an accidental overlap between splits is impossible and the band is legible in the run key: **tune** from 1000 (hyperparameter selection only, never eval or hold-out), **train** from 2000, **val** from 3000, **test** from 4000. The canonical seed 0 belongs to no split. Split instances carry a positive `placement_jitter` scaled to the world size, so every instance differs while the family's grammar holds — train and test are demonstrably the same task, which is exactly what a generalization claim needs and what unconstrained placement would destroy by widening the difficulty distribution instead of sampling from it.

Each split ships as a pinned manifest in the format of `docs/manifest.csv`, one row per instance with its canonical configuration, certified topology, turn-aware optimal $d^\star$, and horizon. Recording $d^\star$ is what makes the splits *checkable*: anyone can confirm that train, val, and test have comparable difficulty distributions rather than taking it on faith.

Two further splits answer different questions and are reported separately, never blended into the headline score: **size extrapolation** (train at the smaller sizes, test at 200 and 400 — the `Chambers2` family exists for precisely this) and **family hold-out**, where whole families are withheld to measure concept transfer.

Inspecting a split. `scripts/browse.py` renders any environment across a seed range or a whole split as a contact sheet, labelling each panel with its seed, $d^\star$, and horizon; `--play` cycles seeds and families interactively.

```
python scripts/browse.py TopoGym/Decoys4-50-v0 --split train -n 12
python scripts/browse.py --all --split tune -n 4 --chunk 5 --panel 400
```

Figures 5–13 are that command's output: every family at four tuning seeds, which is what the splits actually look like.

6.2 Skill ladders (in development)

This subsection describes work in progress: the ladder datasets are not yet generated, and nothing here is pinned. The comparative splits of Section 6 are the design of record; ladders extend them and will be specified once their rungs and difficulty ordering are fixed.

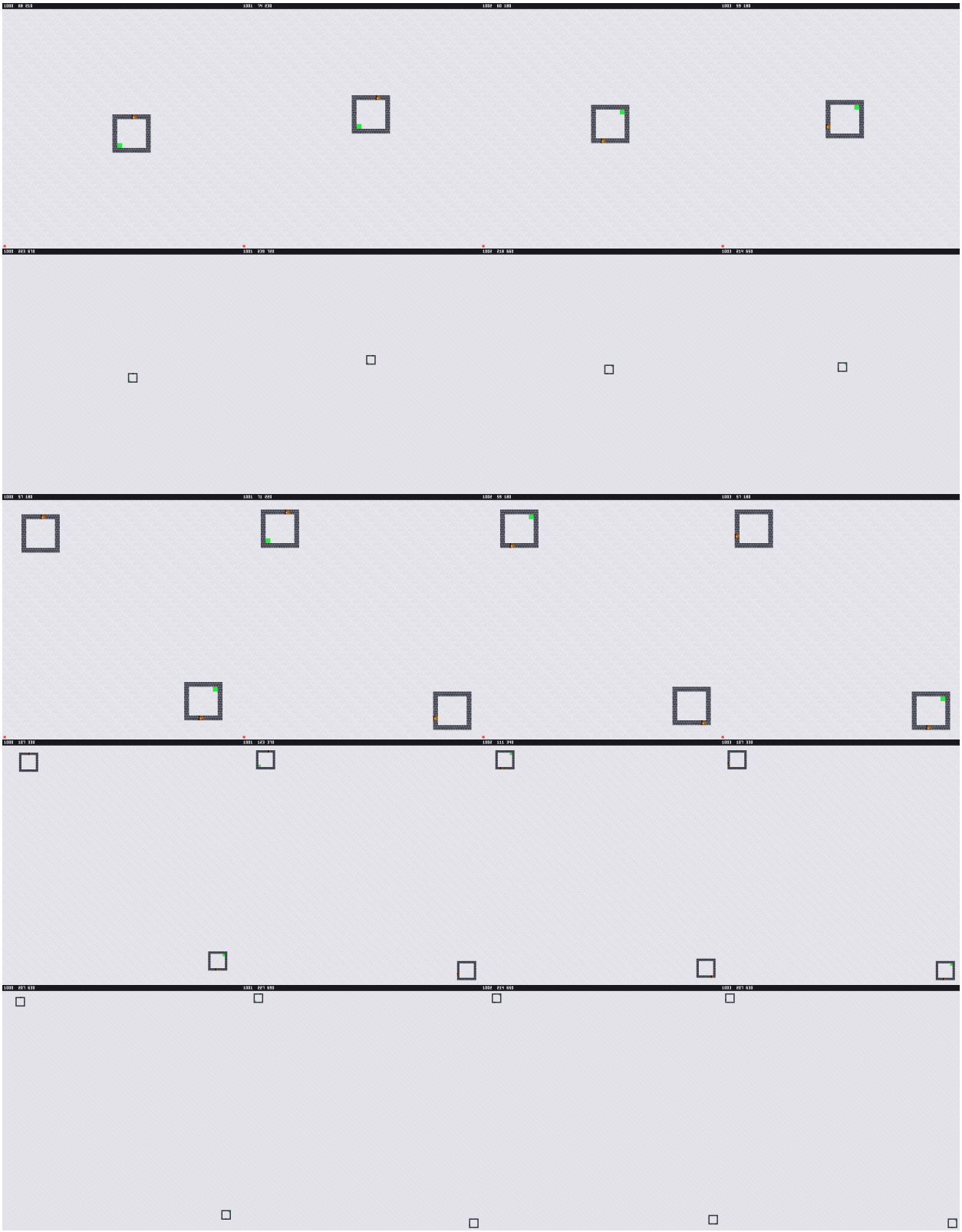

Figure 5: The hyperparameter-tuning (**tune**) split, part 1 of 9: four instances of each family at seeds 1000–1003 with size-scaled placement jitter. Each panel is labelled with its seed, turn-aware optimal $d^\star$, and horizon. Instances differ while the family's arrangement holds — and the $d^\star$ values along a row show the difficulty distribution the split is drawn from.

Figure 6: The hyperparameter-tuning (**tune**) split, part 2 of 9: four instances of each family at seeds 1000–1003 with size-scaled placement jitter. Each panel is labelled with its seed, turn-aware optimal $d^\star$, and horizon. Instances differ while the family's arrangement holds — and the $d^\star$ values along a row show the difficulty distribution the split is drawn from.

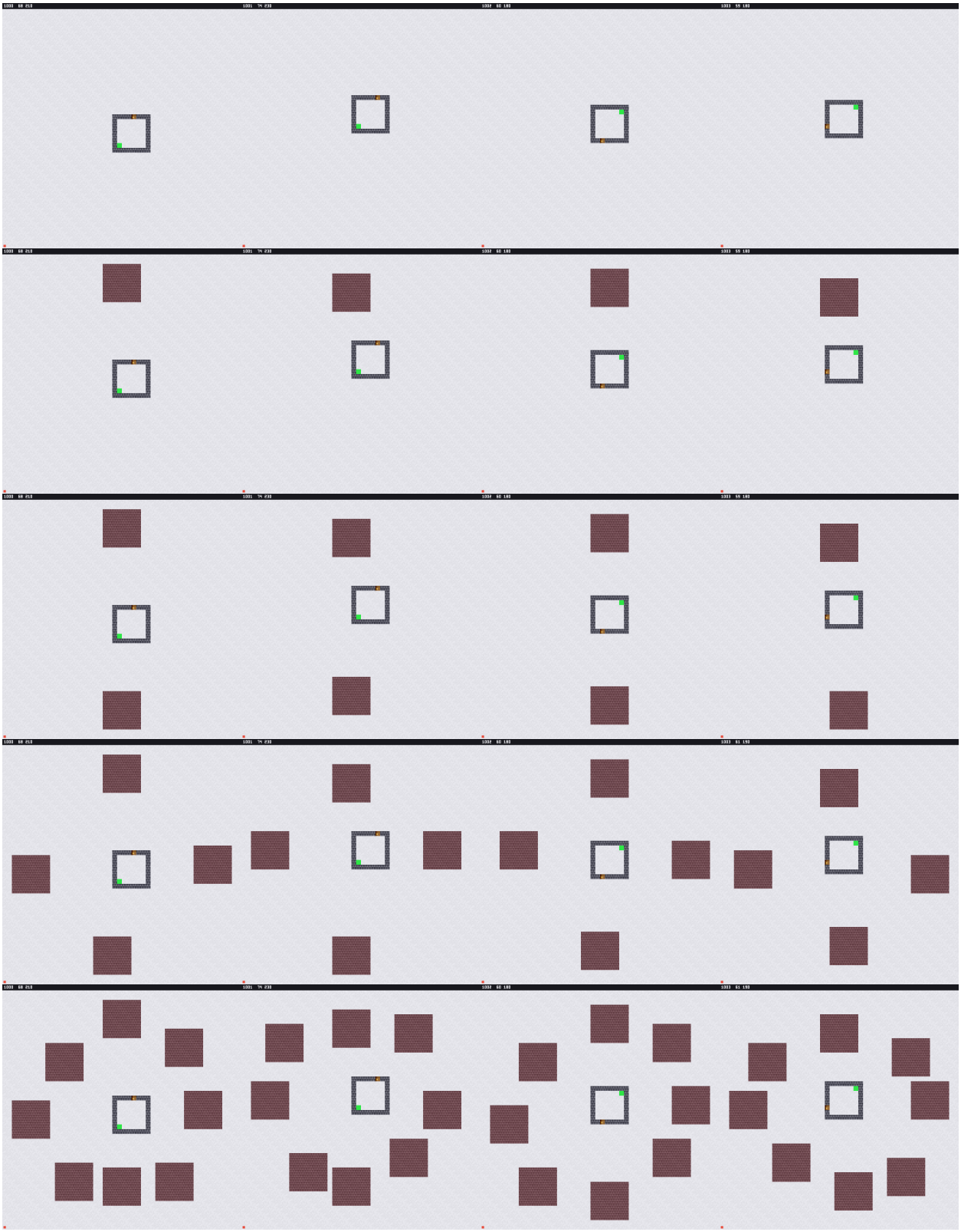

Figure 7: The hyperparameter-tuning (**tune**) split, part 3 of 9: four instances of each family at seeds 1000–1003 with size-scaled placement jitter. Each panel is labelled with its seed, turn-aware optimal $d^\star$, and horizon. Instances differ while the family's arrangement holds — and the $d^\star$ values along a row show the difficulty distribution the split is drawn from.

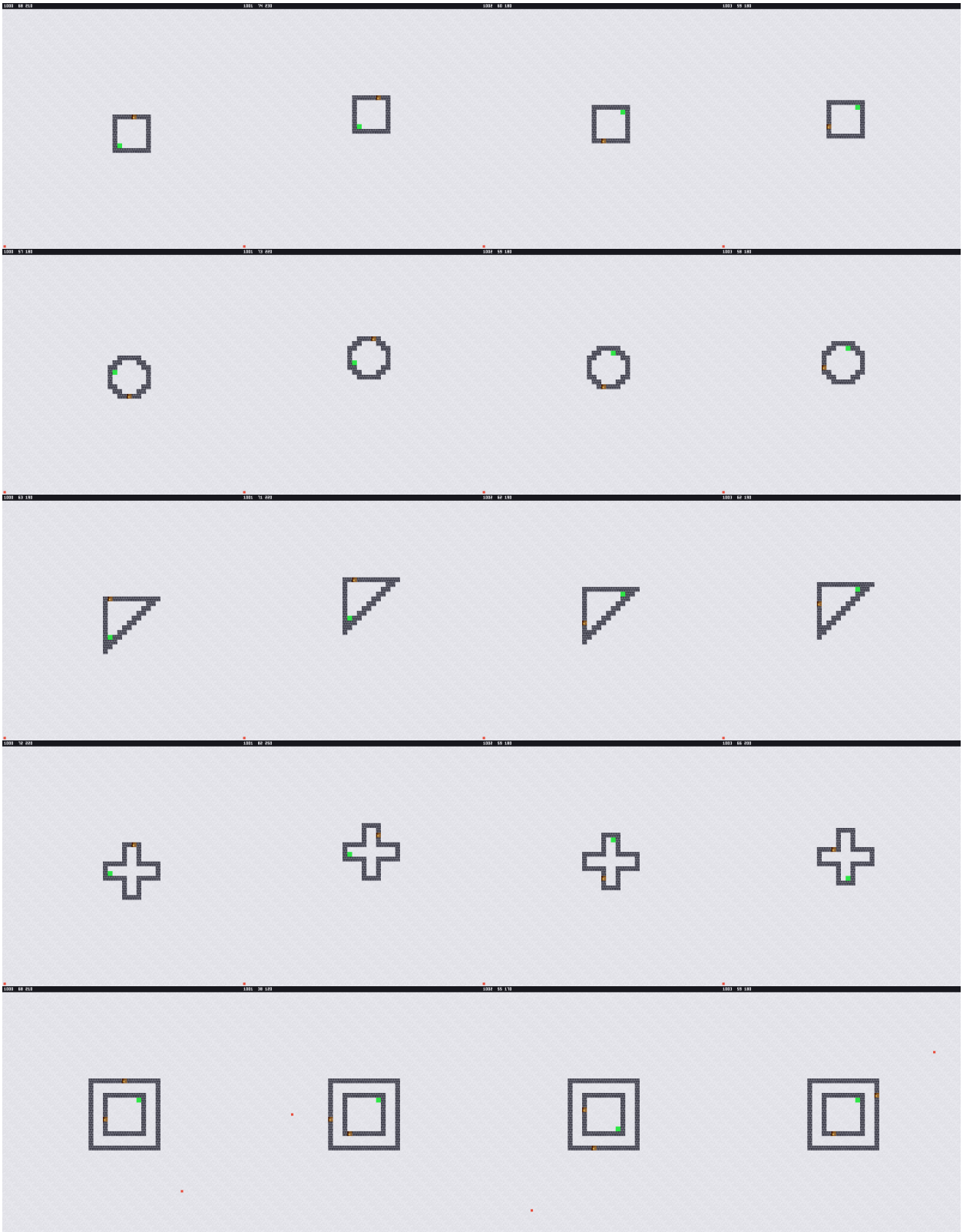

Figure 8: The hyperparameter-tuning (**tune**) split, part 4 of 9: four instances of each family at seeds 1000–1003 with size-scaled placement jitter. Each panel is labelled with its seed, turn-aware optimal $d^\star$, and horizon. Instances differ while the family's arrangement holds — and the $d^\star$ values along a row show the difficulty distribution the split is drawn from.

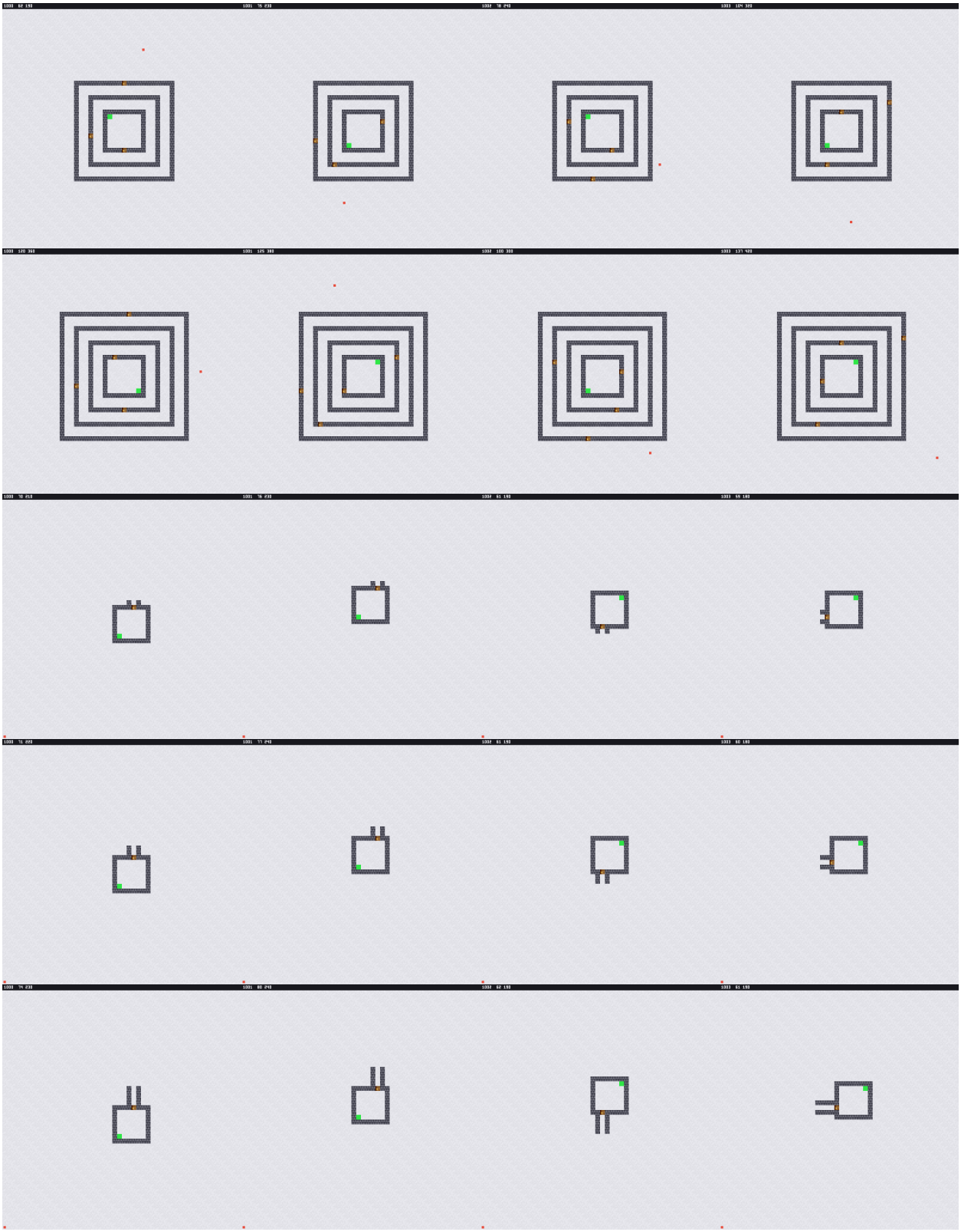

Figure 9: The hyperparameter-tuning (**tune**) split, part 5 of 9: four instances of each family at seeds 1000–1003 with size-scaled placement jitter. Each panel is labelled with its seed, turn-aware optimal $d^\star$, and horizon. Instances differ while the family's arrangement holds — and the $d^\star$ values along a row show the difficulty distribution the split is drawn from.

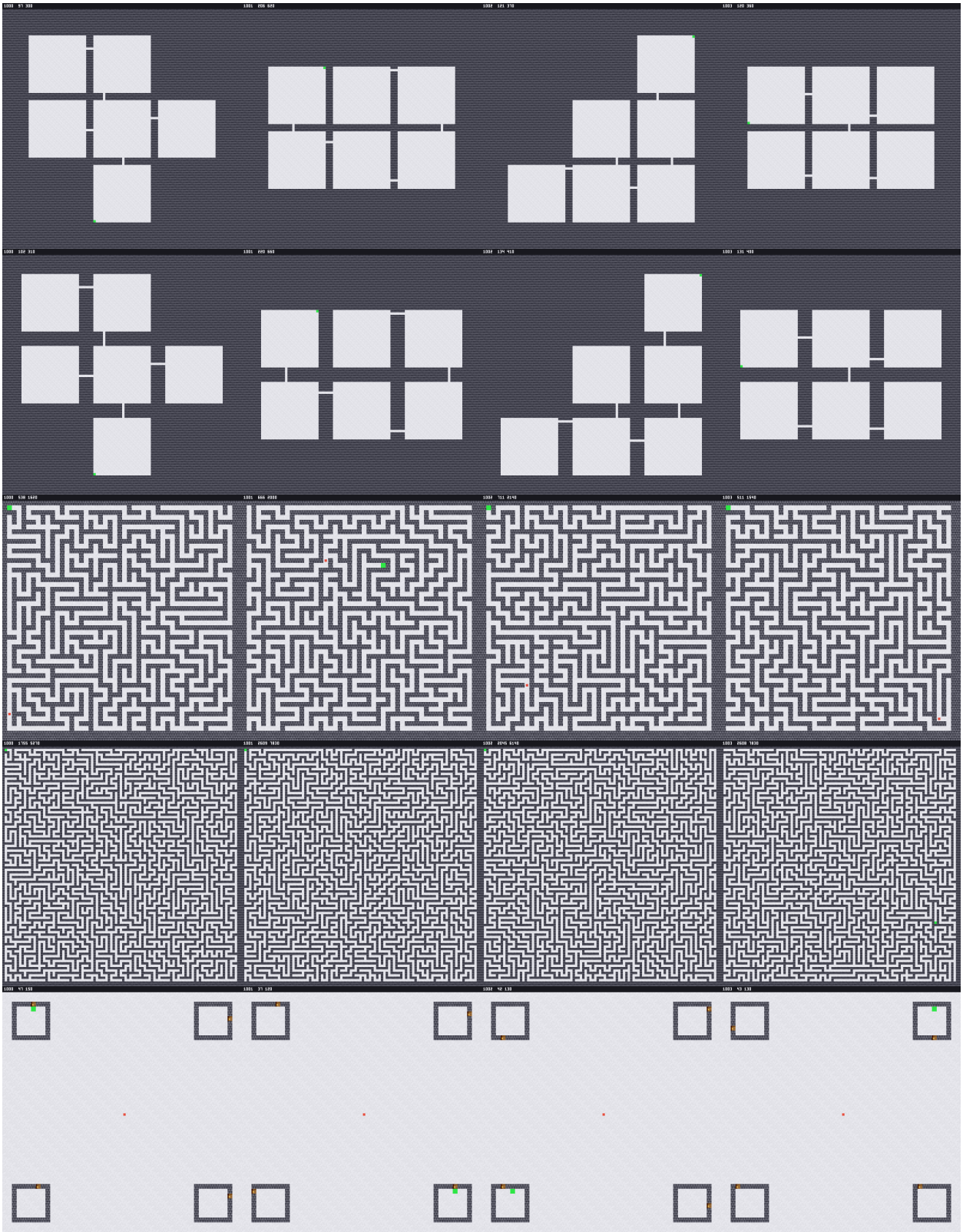

Figure 10: The hyperparameter-tuning (**tune**) split, part 6 of 9: four instances of each family at seeds 1000–1003 with size-scaled placement jitter. Each panel is labelled with its seed, turn-aware optimal $d^\star$, and horizon. Instances differ while the family's arrangement holds — and the $d^\star$ values along a row show the difficulty distribution the split is drawn from.

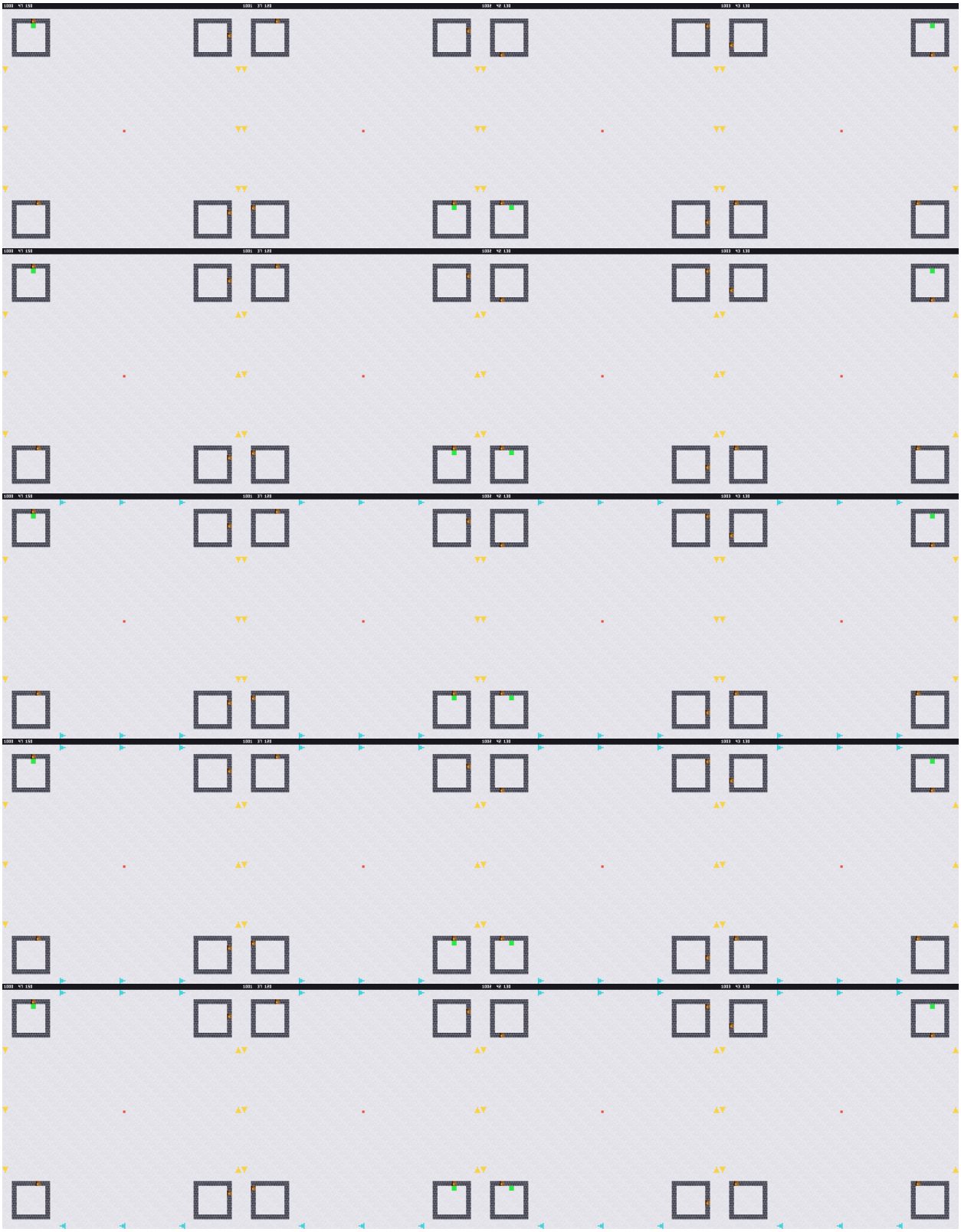

Figure 11: The hyperparameter-tuning (**tune**) split, part 7 of 9: four instances of each family at seeds 1000–1003 with size-scaled placement jitter. Each panel is labelled with its seed, turn-aware optimal $d^\star$, and horizon. Instances differ while the family's arrangement holds — and the $d^\star$ values along a row show the difficulty distribution the split is drawn from.

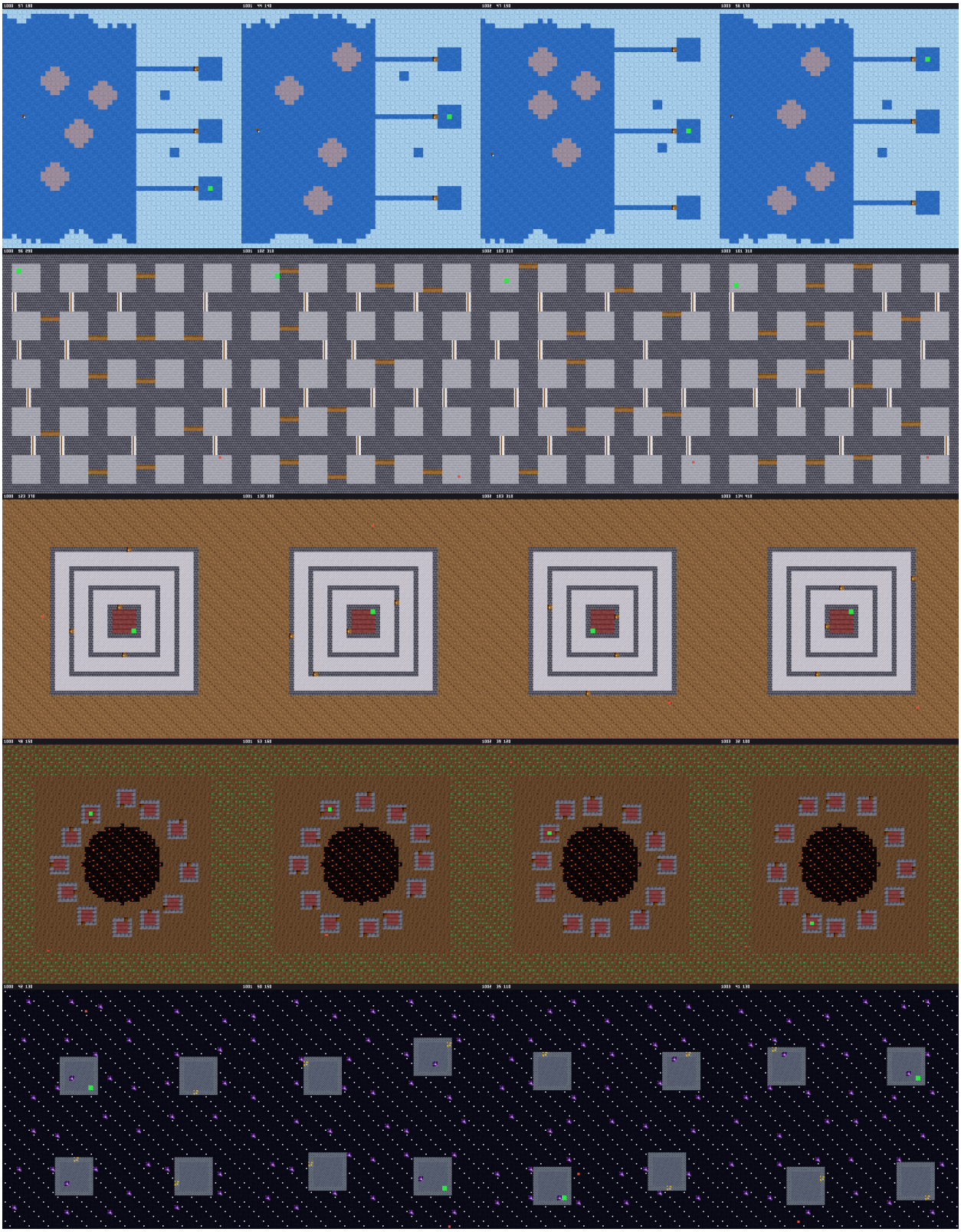

Figure 12: The hyperparameter-tuning (**tune**) split, part 8 of 9: four instances of each family at seeds 1000–1003 with size-scaled placement jitter. Each panel is labelled with its seed, turn-aware optimal $d^\star$, and horizon. Instances differ while the family's arrangement holds — and the $d^\star$ values along a row show the difficulty distribution the split is drawn from.

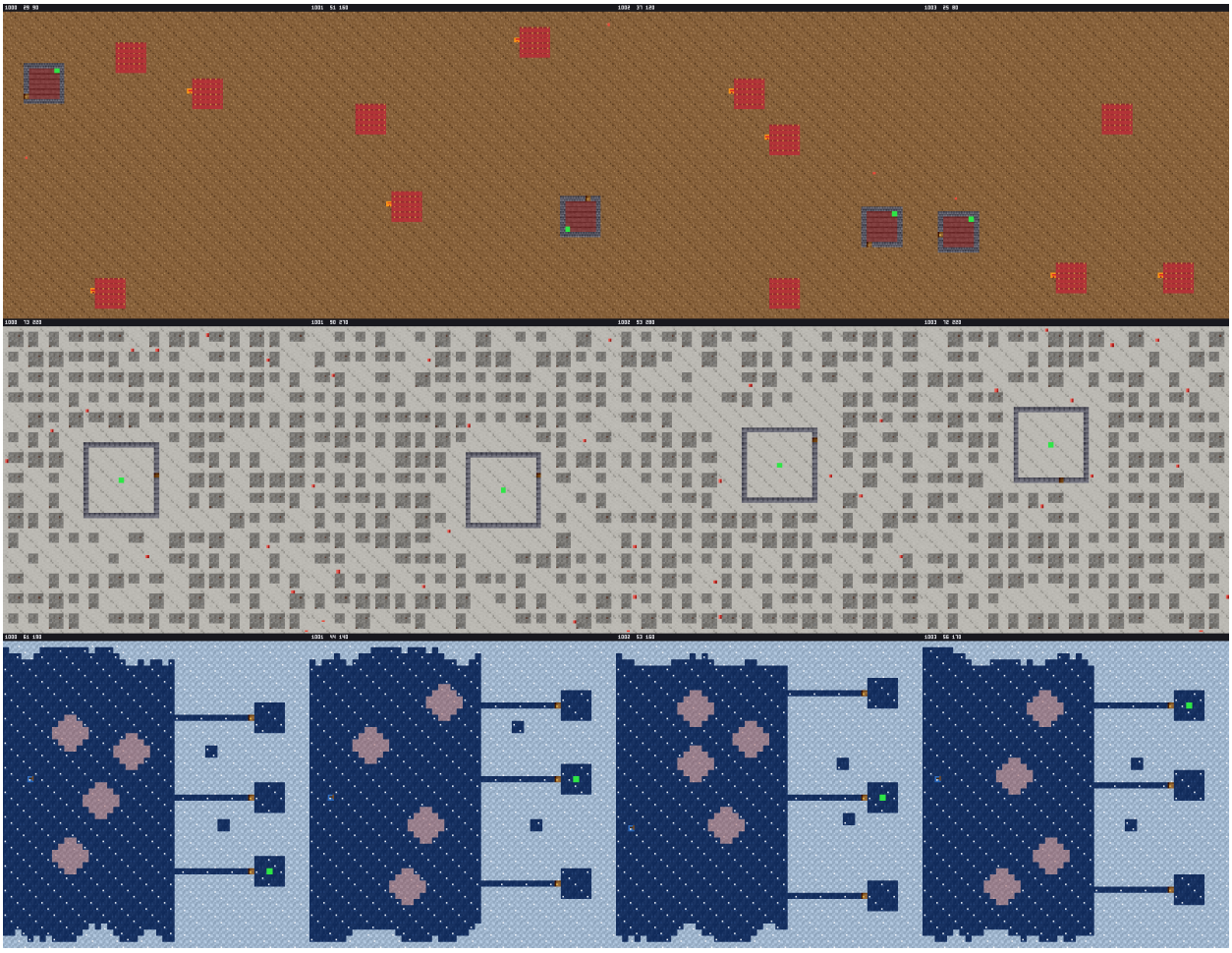

Figure 13: The hyperparameter-tuning (**tune**) split, part 9 of 9: four instances of each family at seeds 1000–1003 with size-scaled placement jitter. Each panel is labelled with its seed, turn-aware optimal $d^\star$, and horizon. Instances differ while the family's arrangement holds — and the $d^\star$ values along a row show the difficulty distribution the split is drawn from.

The intent: per skill (e.g. hole counting, chamber discrimination, decoy rejection, seasonal replanning, hazard routing), a *ladder* dataset of incrementally harder environments with train / eval / test splits drawn from the same seed bands. Ladder *train* splits are ordered — the sequence is part of the dataset, since curricula depend on it — while eval and test are unordered sets at held-out difficulty rungs.

6.3 Baselines

Reference runs on every split and ladder, so each dataset's difficulty is legible: PPO [7] as the plain policy baseline; the exploration family — ICM [8], RND [9], and an archive method in the Go-Explore line [10] consuming `VisitedComplex` cycles (Section 4.4); and GRPO [12] for group-relative training. Reported numbers follow the reliable- evaluation protocol of Agarwal et al. [11] (`rliable`): per-run scores come from the library's deterministic `metrics()` facade, and aggregates across runs, seeds, and environments are interquartile means with stratified bootstrap confidence intervals and performance profiles — never bare point estimates. Benchmark tooling is not part of the core install: `pip install topogym[benchmarks]` pulls the extras (Ray [13] for distributed runs, `rliable` for the aggregate statistics); the core library never depends on them.

References

[1] M. Towers et al. Gymnasium: A Standard Interface for Reinforcement Learning Environments. arXiv:2407.17032, 2024.

[2] C. Maria, J.-D. Boissonnat, M. Glisse, and M. Yvinec. The GUDHI Library: Simplicial Complexes and Persistent Homology. *ICMS 2014*, LNCS 8592, pp. 167–174, 2014.

[3] V. de Silva and G. Carlsson. Topological estimation using witness complexes. *Eurographics Symposium on Point-Based Graphics*, pp. 157–166, 2004.

[4] G. Carlsson. Topology and data. *Bulletin of the AMS*, 46(2):255–308, 2009.

[5] A. Zomorodian and G. Carlsson. Computing persistent homology. *Discrete & Computational Geometry*, 33(2):249–274, 2005.

[6] Y. Ollivier. Ricci curvature of Markov chains on metric spaces. *Journal of Functional Analysis*, 256(3):810–864, 2009.

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.

[8] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell. Curiosity-driven Exploration by Self-supervised Prediction. *ICML 2017*, pp. 2778–2787, 2017.

[9] Y. Burda, H. Edwards, A. Storkey, and O. Klimov. Exploration by Random Network Distillation. *ICLR 2019*, 2019.

[10] A. Ecoffet, J. Huizinga, J. Lehman, K. O. Stanley, and J. Clune. First return, then explore. *Nature*, 590:580–586, 2021.

[11] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare. Deep Reinforcement Learning at the Edge of the Statistical Precipice. *NeurIPS 2021*, 2021.

[12] Z. Shao et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300, 2024.

[13] P. Moritz et al. Ray: A Distributed Framework for Emerging AI Applications. *OSDI 2018*, pp. 561–577, 2018.